

Daniil Martynov

Embedded Systems Engineer (Rust / C)

Developing bare-metal microcontroller firmware in Rust (RTIC) with a strong focus on reliability and memory safety. Designing MCU peripheral schematics in KiCad. Proficient in direct register manipulation and low-level peripheral configuration regardless of the software stack. Currently expanding my toolkit with C (HAL / LL) to align with industry standards.

CONTACTS

Location: Moscow

Employment: Part-time / Internship

GitHub: github.com/rynielic

Telegram: @gyniael1ccc

Email: rynielic@gmail.com

EDUCATION

RTU MIREA, Moscow

Cyber-Physical Systems · 2025 - 2029 · Full-time (2nd Year B.Sc.)

Major: Computer Science and Engineering / Cyber-Physical Systems.

SKILLS & STACK

Microcontrollers & Peripherals:

STM32 · ARM Cortex-M · DMA (Circular) · UART / USART · I2C · 1-Wire · CRC · PWM / Timers · GPIO / EXTI

Languages & Frameworks:

Rust · RTIC · PAC / Raw Registers · C / C++ · Python

Tools & Environment:

Linux · Git · OpenOCD · GDB (arm-none-eabi) · probe-rs · Unit Testing · STM32Cube · Cortex-Debug · KiCad · Soldering & Prototyping · KOMPAS-3D (CAD)

PROJECTS & EXPERIENCE

Non-Blocking UART Parser (DMA, CRC16) for Hardware PWM Power Control

Pet Project (Firmware) · STM32F3DISCOVERY · Rust (no_std) · 2026 · GitHub

Implemented non-blocking variable-length command reception via a circular DMA buffer and USART IDLE interrupt, calculating received byte counts using the hardware DMA counter. Secured the command protocol with hardware CRC-16 validation. Controlled the power load via hardware PWM on TIM3. Configured an emergency stop system triggered by a hardware EXTI0 interrupt and a STOP command with explicit RESUME recovery. Placed the core into sleep mode (WFI) during the idle loop to save power while maintaining hardware-driven data reception and PWM generation. Implemented direct register access (PAC) to bypass HAL architectural limitations. Wrote unit tests for the parser logic. Designed the schematic and soldered a low-side MOSFET (IRLB8721) power switching stage.

Fault-Tolerant Multithreaded Serial/UART Telemetry CLI Tool

Tooling (serial_host) · Linux / Windows · Rust (std) · 2026 · GitHub

Developed a native host client (Linux/Windows) for bidirectional microcontroller communication. Built a concurrent architecture featuring independent read and write threads with safe state synchronization via `AtomicBool` primitives. Integrated software CRC-16 calculation matching the STM32 hardware unit for host-side packet pre-validation. Implemented fault-tolerant I/O error handling (such as physical link disconnection) with automatic serial port hot-plug reconnection without restarting the utility.

PID Temperature Control System

Coursework Project · STM32F3DISCOVERY · Rust · 2026 · GitHub

Developed a closed-loop liquid temperature control system based on an STM32 microcontroller. Implemented a bit-banged 1-Wire interface to poll a DS18B20 sensor with CRC-8 packet validation. Combined a

mathematical PID controller with a bang-bang mode for rapid heating during large temperature errors. Regulated heater power (12V, 50W) using time-proportional control via an electromechanical relay. Enabled live tuning of PID gains and setpoints via UART commands without halting system operation. Implemented telemetry output to an OLED display (I2C) and serial terminal. Designed the circuit schematic in KiCad.

ADDITIONAL INFORMATION

Firmware Architecture: Developing event-driven, non-blocking code based on hardware interrupts (NVIC), timers, and DMA. Reducing power consumption and CPU load by placing the MCU into sleep mode (`WFI`). Enforcing strict memory control without heap allocation (static allocation) and safe real-time concurrency (RTIC).

Hardware & Software Debugging: On-target step-by-step debugging via SWD (ST-LINK, OpenOCD + GDB, probe-rs, RTT logging). Writing companion Python scripts for PC-side telemetry collection and analysis (PySerial). Hardware troubleshooting, continuity testing, signal level verification with a multimeter, and manual perfboard assembly.

Documentation & Standards: Proficient in reading technical documentation in English (Datasheets, Reference Manuals, Errata, B2 level). Experienced in preparing hardware and software engineering documentation: circuit schematics, Bills of Materials (BOM), explanatory notes, and technical specifications.